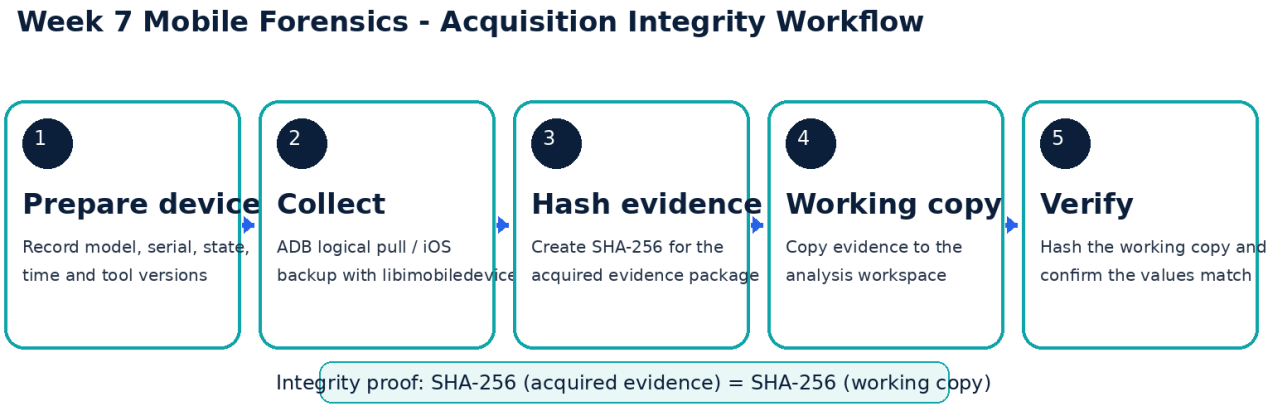
WEEK 7

Mobile Forensics & Malware Triage

Acquisition / Collection Report

Logical vs File-System vs Physical Acquisition on a Test Device



Prepared by	Imtiaz Ali
Program	BS Software Engineering
Internship	NCERT Digital Forensics Internship 2026
Date	20 August 2026

Important lab note: device identifiers, exact tool versions, acquisition timestamps, and SHA-256 values must be copied from the real test device. I have not invented these values in this report.

1. Objective

The purpose of this week was to understand how mobile evidence can be collected in a way that is repeatable, documented, and easy to verify later. I focused mainly on the acquisition stage rather than deep analysis. The practical side uses Android Debug Bridge (ADB) for Android and libimobiledevice for iOS. The main evidence-quality requirement is that every acquisition is logged and that the acquired copy and the working copy produce the same SHA-256 hash. This makes it possible to show that the evidence used for analysis is the same data that was originally collected.

2. Tools and Test Setup

- Android: Android SDK Platform Tools / ADB. Google documents ADB as the command-line bridge used to communicate with a connected device, open a shell, and pull files or directories [1].
- iOS: libimobiledevice. The project provides utilities such as idevicepair, ideviceinfo, and idevicebackup2 for pairing, identifying a device, and creating an iOS backup [3]-[4].
- Hash verification: SHA-256 using PowerShell Get-FileHash on Windows. Microsoft documents that matching file hashes can be used to verify identical content [5].
- Evidence storage: a dedicated evidence folder and a separate working-copy folder. Raw acquisition output should not be edited directly.

For Android, USB debugging must be enabled on the test phone and the RSA authorization dialog must be accepted on the device before ADB can be used. This is a change to device state, so it should be written in the acquisition log [1]. On recent Android systems, file-based encryption limits what can be collected without elevated or specialized access; devices launching with Android 10 and later are required to use file-based encryption [2].

3. Acquisition Types and Trade-offs

Method	Typical coverage	Access needed	Main advantage	Main limitation
Logical	User-accessible files, backups, metadata, app data exposed by OS services	Unlocked / authorized device	Low risk and fast	Does not capture every protected file
File-system	Broader directory and application data	Root, jailbreak, privileged agent, or specialist method	More context than logical	Access method may alter the device and is often restricted
Physical	Block-level / raw storage representation	Device-specific exploit, bootloader, chip-level or specialist hardware	Potentially widest coverage	Modern encryption can make raw data unreadable; higher risk and complexity

For this week, logical acquisition is the most appropriate repeatable method for an ordinary test device because it is supported by open-source tools and has a lower risk of modifying the phone. File-system and physical methods are documented as comparison points. NIST treats preservation, acquisition, examination, analysis, and reporting as separate parts of a sound mobile-forensics process [6].

4. Evidence Folder Structure

```
Week7_Mobile_Forensics/
  logs/
  evidence/
    android/
    ios_backup/
    apk_triage/
  working_copy/
  hashes/
  screenshots/
  report/
```

The evidence folder is treated as the original acquisition location. Any examination should be done from working_copy. This simple separation helps avoid accidental changes to the original collected data.

5. Android Logical Acquisition with ADB

The following is the workflow I would use on the authorized Android test device. The exact model, serial number, Android version, and ADB version should be copied into the log before acquisition.

1. Connect the phone with a known USB cable, unlock it, enable USB debugging, and accept the RSA authorization prompt.
2. Record the tool version and connected-device details.
3. Record basic device properties and installed third-party packages.
4. Pull user-accessible storage into the evidence folder. For a smaller acquisition, collect selected folders such as DCIM, Download, and Documents.
5. For malware triage, identify third-party packages and collect a suspicious APK only where the device permits the pull.
6. Stop collection, record end time, file count/size, and calculate a SHA-256 hash of the final evidence package.

```
adb version
adb devices -l
adb shell getprop > logs\android_getprop.txt
adb shell pm list packages -f -3 > logs\third_party_packages.txt
adb pull /sdcard/DCIM evidence\android\DCIM
adb pull /sdcard/Download evidence\android\Download
adb pull /sdcard/Documents evidence\android\Documents
```

For a suspicious application, the package path can be requested first. Pulling from protected locations can fail on a standard unrooted device, and that failure should be logged rather than bypassed without authorization.

```
adb shell pm path com.example.suspect
adb pull <returned_apk_path> evidence\apk_triage\
```

6. iOS Logical Acquisition with libimobiledevice

On iOS, libimobiledevice provides a practical open-source logical-backup workflow. The phone must be connected and trusted by the host. idevicepair manages pairing, ideviceinfo records device

information, and idevicebackup2 creates a backup. The current manual documents the syntax as idevicebackup2 backup --full DIRECTORY [3].

```
idevicepair pair
idevicepair validate
ideviceinfo > logs/ios_device_info.txt
idevicebackup2 -v
idevicebackup2 backup --full evidence/ios_backup
idevicebackup2 info evidence/ios_backup
```

If the iOS backup is encrypted, the password should not be typed directly into a command that will remain in shell history. The idevicebackup2 documentation recommends interactive input or an environment variable instead [3].

7. Malware Triage Collection

The malware-triage part of this week is limited to safe collection. I would not execute a suspicious APK or unknown mobile application. The aim is to preserve material that can later be analyzed in a controlled environment.

- Record the package name, application path, version information, and the date/time of collection.
- Copy the APK or available application artifact only when the test-device permissions allow it.
- Calculate SHA-256 immediately after collection and again after moving the file to the working directory.
- Keep any suspicious sample isolated from normal personal folders and do not upload it to a third-party service unless the case authorization allows data sharing.

8. Integrity Verification

Integrity proof is the most important result for this task. The original acquisition output is hashed first. A working copy is then created and hashed again. The two SHA-256 values must match. PowerShell Get-FileHash computes a file hash and uses SHA-256 by default [5]. For a folder acquisition, I would first create an evidence archive (for example ZIP or TAR) and hash that archive so the verification is simple and repeatable.

```
Get-FileHash .\evidence\android_acquisition.zip -Algorithm SHA256
Copy-Item .\evidence\android_acquisition.zip .\working_copy\
Get-FileHash .\working_copy\android_acquisition.zip -Algorithm SHA256
```

9. Full Acquisition Log

The table below is the final acquisition log format. Fields marked "Record during lab" must be filled from the real test device. They are intentionally not invented.

Case ID	W7-MOB-2026-01
Examiner	Imtiaz Ali
Source device	Record model + serial/UDID during lab
Device state	Record locked/unlocked, battery, network/airplane-mode state

Acquisition method	Logical acquisition
Tool	ADB (Android) / libimobiledevice (iOS)
Tool version	Record output of adb version / idevicebackup2 -v
Start time	Record during lab
End time	Record during lab
Original evidence path	Record final evidence package path
SHA-256 - original	Record generated hash
SHA-256 - working copy	Record generated hash
Integrity result	MATCH only when both recorded hashes are identical

10. Expected Verified Result

Verification is successful when the SHA-256 hash of the original acquisition package exactly matches the SHA-256 hash of the working copy. The actual values must be taken from the lab run and pasted into the acquisition log and presentation.

11. Limitations and Lessons Learned

- Logical acquisition is repeatable and practical, but it does not equal a complete physical image.
- ADB access depends on device authorization and permissions. Protected app-private data normally cannot be collected by simple adb pull on an unrooted device.
- Modern Android encryption reduces the usefulness of raw physical storage when decryption keys are unavailable [2].
- iOS backup content depends on device state, pairing, backup settings, and iOS protections.
- Every action that changes device state - for example enabling USB debugging or trusting a host - should be written in the log.

12. Conclusion

This task showed me that mobile acquisition is not only about copying files. A useful forensic collection needs a clear source, a documented method, tool versions, accurate times, and integrity verification. ADB and libimobiledevice are useful open-source tools for logical acquisition on test devices, but they also show the limits created by permissions and modern encryption. The most important practical result is the repeatable evidence-handling process: collect, preserve, hash, copy, hash again, and confirm a match before analysis begins.

References

[1] Google, "Android Debug Bridge (adb)," Android Developers, updated 2026. https://developer.android.com/tools/adb

[2] Android Open Source Project, "File-based encryption," Android Security documentation, 2026. https://source.android.com/docs/security/features/encryption/file-based

[3] libimobiledevice, "idevicebackup2(1) - Create or restore backups for iOS devices," official project manual. https://github.com/libimobiledevice/libimobiledevice/blob/master/docs/idevicebackup2.1

[4] libimobiledevice, "ideviceinfo(1)" and "idevicepair(1)," official project manuals. https://github.com/libimobiledevice/libimobiledevice/tree/master/docs

[5] Microsoft, "Get-FileHash," PowerShell documentation.
https://learn.microsoft.com/powershell/module/microsoft.powershell.utility/get-filehash

[6] NIST, "Guidelines on Mobile Device Forensics," SP 800-101 Rev. 1, May 2014. https://csrc.nist.gov/pubs/sp/800/101/r1/final

Final Submission Checklist

- Add real screenshot of adb devices -l or ideviceinfo.
- Add real acquisition start/end screenshots.
- Paste the exact tool version output.
- Paste original and working-copy SHA-256 values.
- Confirm both hashes match before submitting.